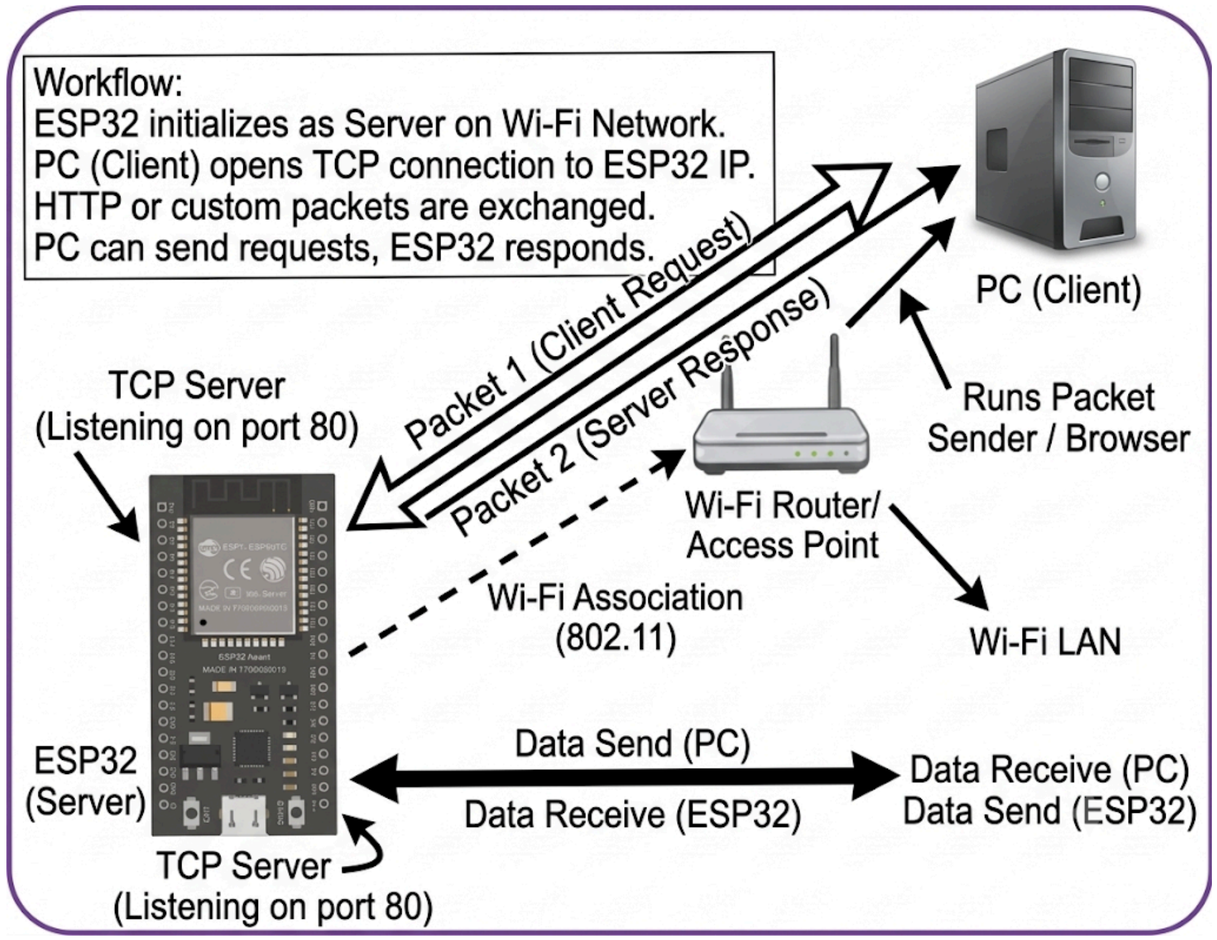
TCP Communication Protocol

Reliable Client-Server Wireless Communication via Local Wi-Fi.

What Is TCP?

A request-response protocol enabling an ESP32 client to fetch data from or send data to web servers using standard Wi-Fi infrastructure.



Rides the Wi-Fi Radio

Uses standard Wi-Fi protocols over a local network/router rather than vendor-specific action frames.

Access Point Needed

Requires a Wi-Fi Access Point/Router to route packets between the PC and ESP32

Connection-Oriented

Reliable Stream Data, utilizing TCP sockets for verified packet delivery.

Flexible Testing

Allows direct communication with testing software like Packet Sender on a PC.

Where Is TCP Used in IoT?

Replace peer-to-peer/sensor mesh cases

Local Web & Control Servers

Real-Time PC-to-ESP Telemetry & Data Logging

Home Automation Hubs requiring reliable IP networking

Bi-directional Request-Response Services

Prerequisites & Hardware :

Hardware: 1x ESP32 Development Board, 1x Micro-USB/USB-C Data Cable, and 1x PC connected to a 2.4 GHz Wi-Fi network.

USB data cable: Required for programming and Serial Monitor access. Make sure the cable supports data, not only charging.

Power: USB power is sufficient for normal testing. A separate regulated supply can be used if the board's power-input requirements are satisfied.

Software: Update to include Arduino IDE 2.x and Packet Sender (installed via CLI using winget install packetsender)

Network Requirements: A 2.4 GHz Wi-Fi Network (SSID and Password)

Step 1: Configure the Arduino IDE

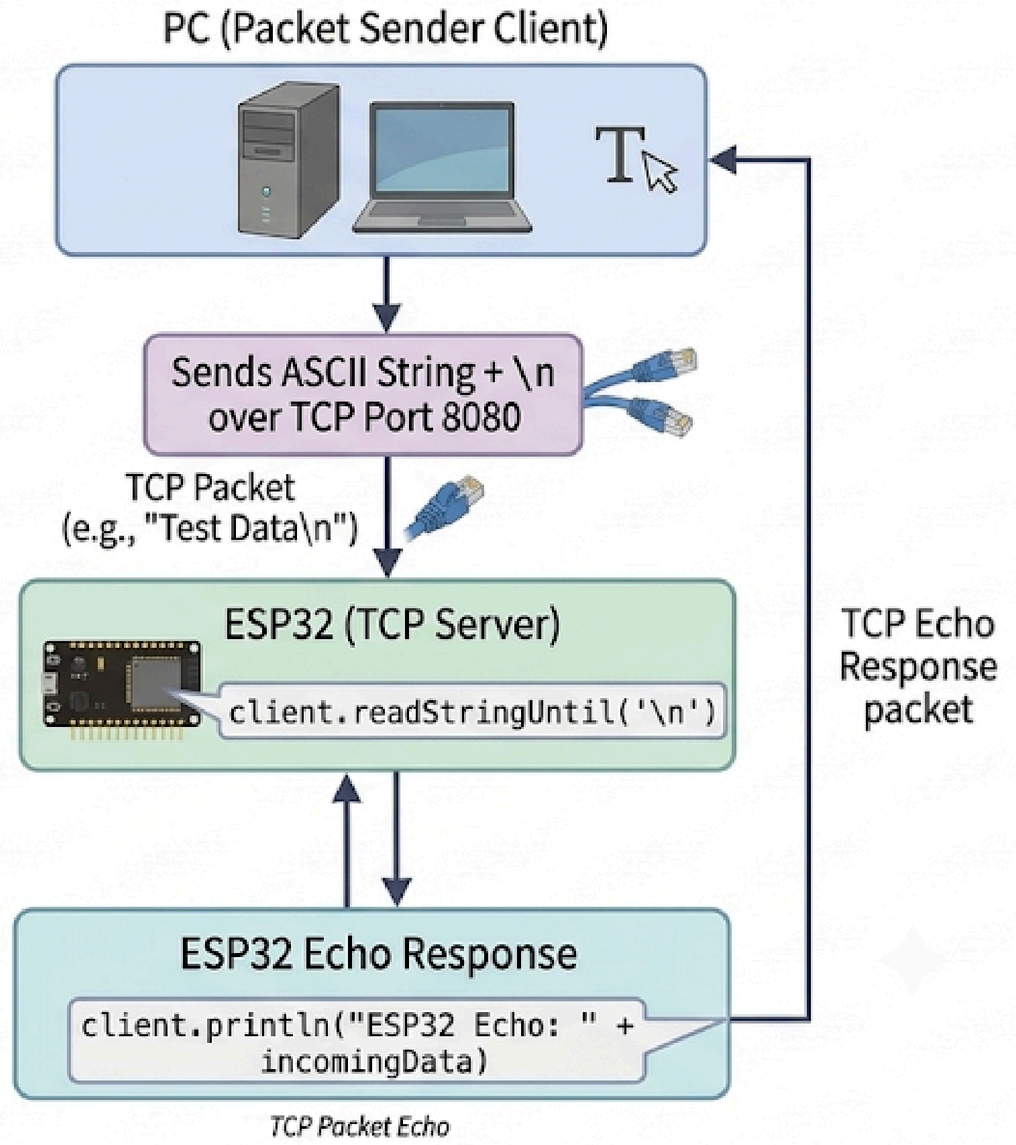
Step 2: Configure & Upload ESP32 TCP Server Code

Step 3: Setting Up Packet Sender (PC Client)

Step 4: Perform an HTTP POST Request (Send Data/Telemetry)

Step 5: Run Trial & Verify Communication

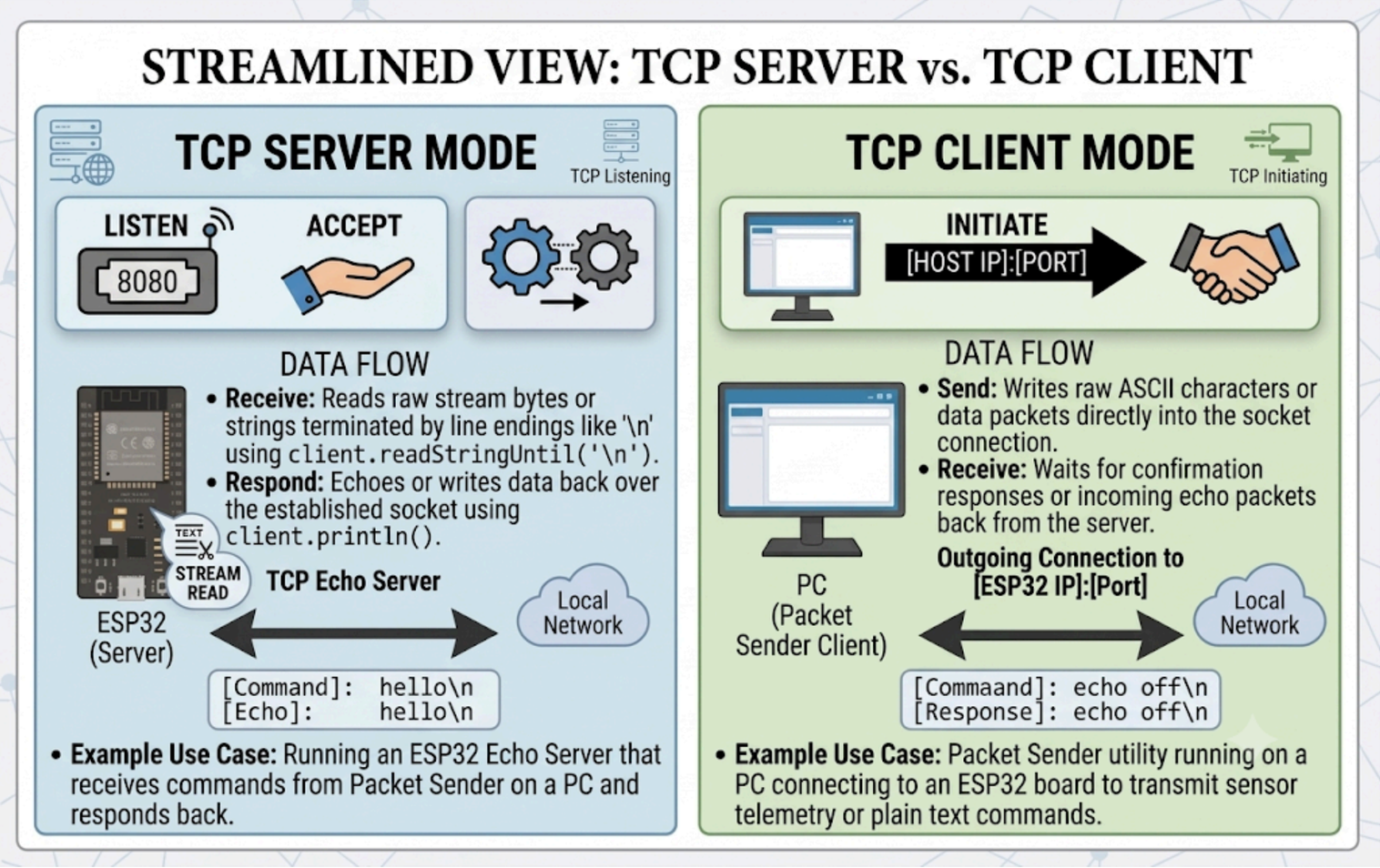
Understand the TCP Packet Eco Flow



TCP Packet Echo

TCP Communication Roles Comparison

TCP Server listens for incoming client connections on a specified local port, while a TCP Client initiates a connection to a target IP address and port to send or request raw stream data.



A TCP connection relies on exact socket endpoints defined by an IP address and Port number (e.g., 192.168.1.50:8080) rather than HTTP URL paths. Ensure client transmissions include the expected line-ending delimiter (such as \n or \r\n), as socket reading methods depend on it to mark the end of a message buffer.

That's TCP Communication!

The access the implementation guide
Scan the QR / click on the link in the description

Thank You